StackSense Challenge

Intelligent Tech Stack Recommendation Engine - Project Report

Full name	Nitish Prabagaran
College	Panimalar Engineering College, Chennai (B.E. Computer Science, 2025-2029)
Unstop registered email	nitishprabagaran07@gmail.com
GitHub	https://github.com/NITISH-027/stacksense26
Event	Synergy '26, International Institute of Information Technology (IIIT), Bangalore
Date	14 November 2026

1. Abstract

StackSense is a deterministic recommendation engine that turns a structured project description - scale, team, budget, infrastructure, compliance and priorities - into a complete, justified technology stack: frontend, backend, database, hosting and the supporting services the inputs actually call for. Selection is done by hard-constraint filtering (data residency, compliance regimes, deployment model and cloud preference are never violated silently) followed by weighted priority scoring over a curated knowledge base of real, currently maintained technologies. Every component ships with a justification tied to named input fields, at least one rejected alternative with the reason it lost, honest trade-offs, and an order-of-magnitude monthly cost band. The engine validates its input field by field, flags contradictory constraints with severities instead of crashing, and is fully reproducible: identical input always produces an identical stack, with no LLM involved in any decision.

2. Problem understanding

The challenge is not to name popular tools but to argue for the *right* tools for a given situation. I read the task as three sub-problems: (a) capture the inputs that genuinely change architecture decisions - a three-person student team and an RBI-bound fintech must not receive the same answer; (b) encode enough accurate technology knowledge to reason over; and (c) explain every choice so a senior engineer can audit it.

Scope decisions and assumptions: the engine covers the four core layers plus the supporting-service families named in the brief (cache, queue, search, auth, storage, CDN, realtime, payments, ML, CI/CD, observability). 'None' is emitted for a layer only when the requirements genuinely do not need it (a pure content site needs no custom backend or database). Budget, timeline and priorities are treated as soft constraints traded off through scoring; data residency, compliance, deployment model and explicit cloud preference are hard constraints enforced by filtering. Cost estimates are bands with stated assumptions, not quotes. A UI was built because it makes evaluation easier, but the CLI and the JSON in/output contract are the primary interface, since judges run hidden scenario files.

3. System architecture

The engine is a small, dependency-free Python pipeline. Each stage has one job and its output feeds the next; nothing is hidden in a framework.

```
scenario JSON
  |   validate_and_fill() - schema, types, enums, ranges; documented defaults
  |   detect_conflicts() - contradictions, risks, deprecated existing systems
  |   hard-constraint filters - residency, compliance, deployment, cloud
  |   weighted scoring - priorities x attributes + team/scale/budget fit
  |   selection - highest score wins, deterministic tie-break
  |   explanation - justification, driven_by, alternatives, trade-offs
  |   cost model - monthly USD band with assumptions
  v
output JSON (standard schema)
```

Modules: **kb.py** holds the technology knowledge base (candidates per layer with cost profile, operational burden, maturity, compliance support, regions and fit notes, plus a deprecated-technology registry). **engine.py** is the pipeline. **cli.py** exposes recommend / batch / validate / ui commands. **ui_server.py** and **static/index.html** provide the web UI using only the standard library. Data flow is one-directional: a scenario enters as JSON, passes validation and conflict detection, loses ineligible candidates to hard filters, is scored, and exits as the standard output JSON shown verbatim in the UI.

4. Input design and research

The engine accepts every mandatory and optional field of the standard schema, and adds four extra parameters that architecture decisions actually turn on:

• **scale.data_volume_gb_12_months** - storage sizing drives the search choice (PostgreSQL full-text search is plenty up to roughly 100 GB; beyond it a dedicated search service earns its keep) and feeds the storage part of the cost band.

• **team.has_devops** - operational capacity is the silent killer of ambitious stacks; without it, high-burden components (Kubernetes-class hosting, Elasticsearch, Kafka) are penalised and managed platforms are preferred.

• **requirements.p95_latency_ms** - a hard latency target strengthens performance-weighted choices and is surfaced in cost assumptions.

• **infrastructure.existing_systems** - checked against a registry of deprecated/end-of-life technologies (AngularJS, MySQL 5.7, Python 2, Docker Swarm, Moment.js and similar) so migration risk is raised as a warning instead of discovered in production.

Every optional field has a documented default that is recorded verbatim in the output's assumptions array (for example traffic_pattern defaults to 'steady', availability to '99.9'). Validation is field-level and total: wrong types, negative counts, unknown enum values, out-of-range priorities and unknown compliance regimes are all reported together with status invalid_input - the engine never crashes on bad input. Cross-field contradictions are detected separately: huge scale on a zero budget (critical), enterprise availability targets with a beginner solo team (critical), many platforms on a short timeline (warning), offline support on web-only (warning), SaaS-needing features on air-gapped deployments (warning). The research behind the input design: which parameters real architecture reviews ask for was cross-checked against public architecture guidance (AWS Well-Architected Framework, cloud vendor solution-architect blogs) and the consistency of the mandatory schema with the six public scenarios.

5. Knowledge base and decision logic

Knowledge base. Each technology is represented as a record of: attribute scores (cost efficiency, time-to-market, scalability, performance, maintainability, security, each 1-5), operational burden, maturity, primary languages, supported platforms or regions, compliance programmes, self-hostability, project-type fit and a one-line curator's note. Coverage: 12 frontend options (web, static, cross-platform mobile, native mobile, desktop), 10 backend options (including serverless), 7 databases (relational, document, key-value, embedded), 11 hosting options (PaaS, the three major clouds, Firebase, VPS, on-premise) and 2-5 options per supporting layer. Attributes were curated from official documentation and pricing pages, the Stack Overflow Developer Survey and DB-Engines (Section 10). Only real, maintained products are included; deprecated technology appears only in the registry used to warn about existing systems.

Decision logic. Selection is filtering plus scoring. Hard constraints remove candidates first: on-premise or air-gapped deployment keeps only self-hostable components; a data-residency requirement keeps only hosting providers with a region in that geography (RBI localisation forces India); each compliance regime keeps only providers that support it; an explicit cloud preference keeps only that cloud. Static-only platforms (Vercel, Netlify, Firebase Hosting) are removed when the project needs a long-running backend server, and Lambda-style hosting is removed for realtime workloads, because WebSockets need long-lived connections.

Scoring. Survivors are scored as a weighted sum - each priority (cost, time-to-market, scalability, performance, maintainability, security, weighted 1-5 by the scenario) times the candidate's matching attribute - plus contextual bonuses and penalties computed from the actual inputs: team-language match, operational burden versus team experience and DevOps capacity, project-type fit, scale appropriateness (scale machinery a tiny project does not need is penalised as over-engineering), realtime support, SEO capability for SEO-critical sites and free-tier fit for zero budgets. The highest total wins; ties break by maturity, then alphabetically, so the result is fully reproducible. Budget is checked afterwards against the cost band: overshoot raises a NEAR_BUDGET warning or an OVER_BUDGET critical, and a zero budget at large scale produces an infeasible status with the relaxed constraint named in assumptions.

Hard vs soft constraints. Data residency, compliance, deployment model and an explicit cloud preference are hard: violating candidates never appear. Budget, timeline and priorities are soft: they move

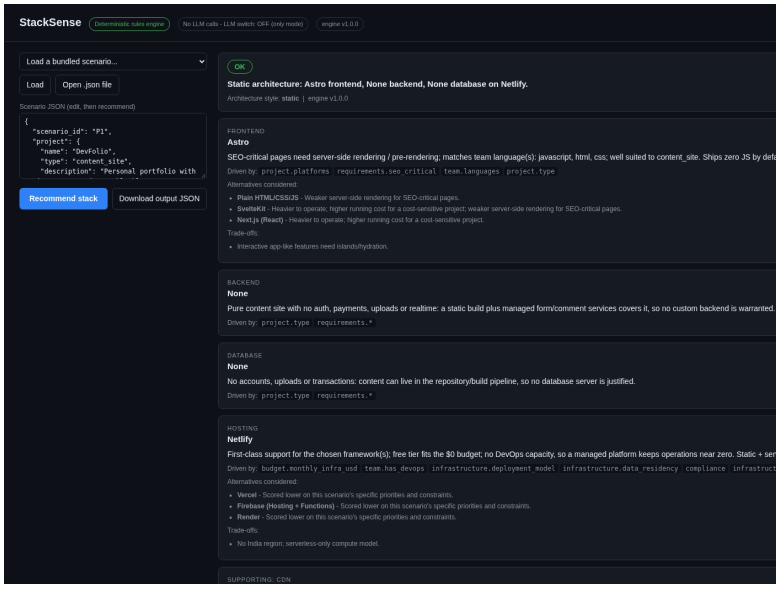
scores and raise warnings, and when they cannot all be met the engine says so (status infeasible or ok_with_warnings with the relaxation stated) instead of answering confidently.

Determinism. There is no randomness, no clock dependence and no iteration-order dependence anywhere in the pipeline; candidate order in the knowledge base is fixed and sorting is by explicit keys. The six public scenarios were each run twice and the outputs were byte-identical.

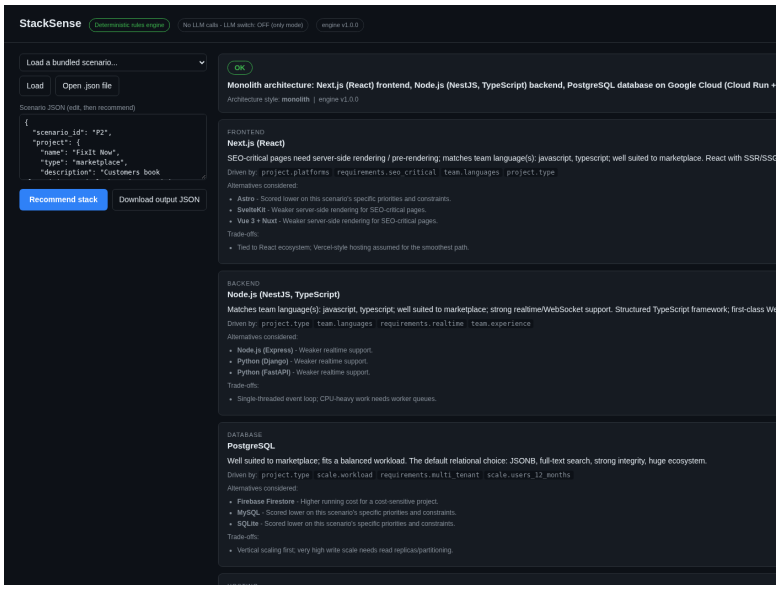
LLM usage declaration and LLM-off switch. The engine makes no LLM calls at all - not for selection, not for parsing, not for wording. There is therefore nothing to disable, and the LLM-off test passes trivially: the stack is identical in every configuration. To make the contract explicit, the CLI accepts '--llm off' (the only value it accepts) and documents this in the README. AI assistance used while building the engine itself is declared in Section 11.

6. Sample recommendations

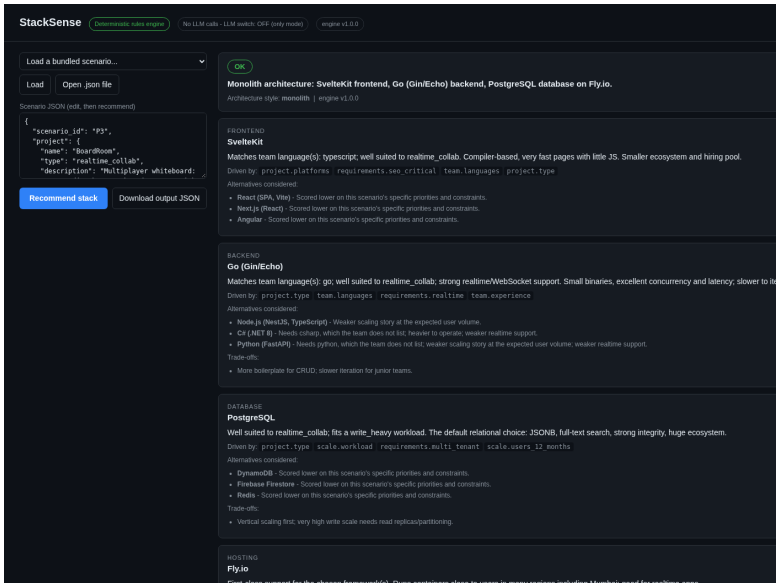
All six public scenarios were run through the engine; the outputs below are the actual files in the submission's outputs/ folder, and every screenshot shows the engine's real UI serving those results. P2 and P4 are walked through in depth.



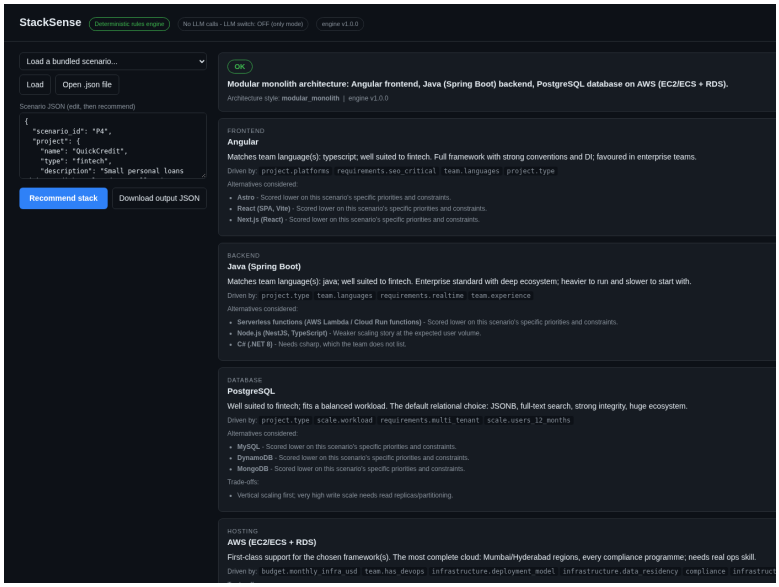
P1 - Personal portfolio + blog: Astro, no backend, no database, Netlify. $0-5/month.



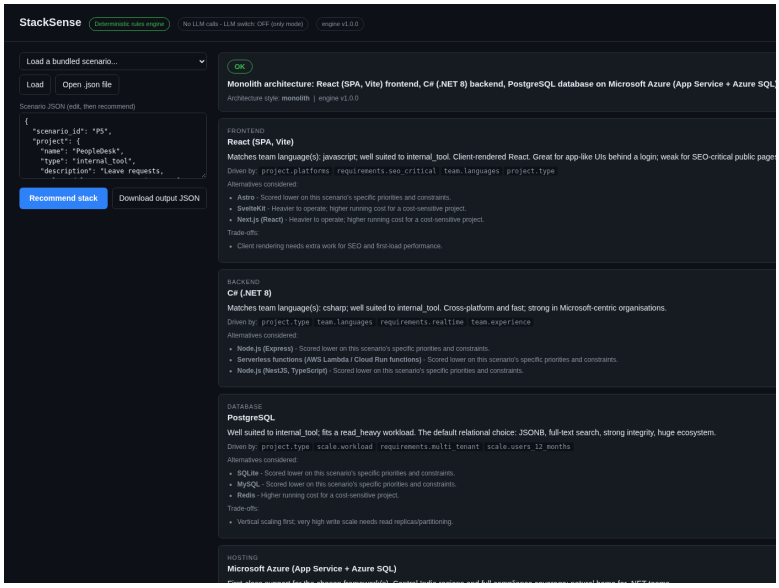
P2 - Home-services marketplace (India): Next.js, NestJS, PostgreSQL, Google Cloud (Mumbai).



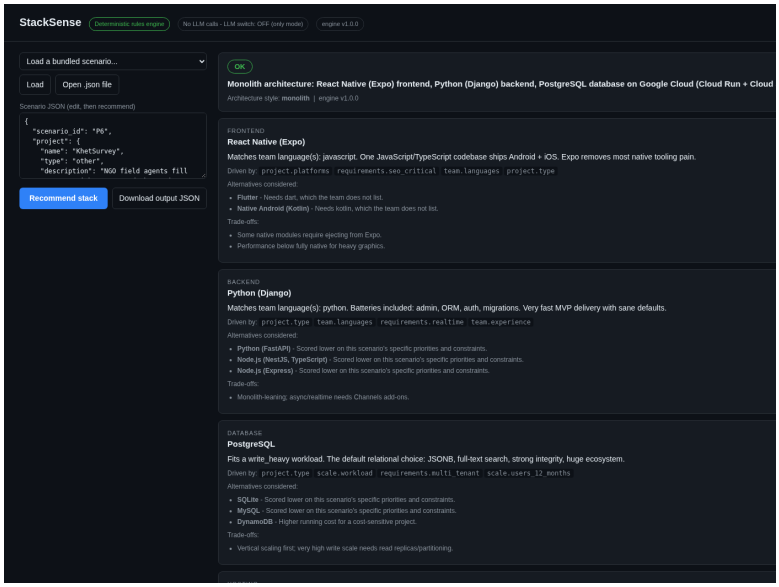
P3 - Realtime whiteboard SaaS: SvelteKit, Go, PostgreSQL, Fly.io, custom WebSocket service.



P4 - Digital lending (India): Angular, Spring Boot, PostgreSQL, AWS (RBI/PCI/ISO compliant regions).



P5 - Internal HR tracker: React SPA, .NET 8, PostgreSQL, Azure (Central India), Keycloak SSO.



P6 - Offline crop survey: React Native, Django, PostgreSQL, GCP, on-device SQLite + sync.

6.1 Walkthrough: P2 - FixIt Now (home-services marketplace, India)

Status: **ok** - Monolith architecture: Next.js (React) frontend, Node.js (NestJS, TypeScript) backend, PostgreSQL database on Google Cloud (Cloud Run + Cloud SQL). Estimated cost: $102-$624/month.

Layer	Choice	Why (from engine output)
frontend	Next.js (React)	SEO-critical pages need server-side rendering / pre-rendering; matches team language(s): javascript, typescript; well suited to marketplace. React with SSR/SSG; the most adopted React meta-framework. Strong SEO and ec...
backend	Node.js (NestJS, TypeScript)	Matches team language(s): javascript, typescript; well suited to marketplace; strong realtime/WebSocket support. Structured TypeScript framework; first-class WebSocket support and a huge ecosystem.
database	PostgreSQL	Well suited to marketplace; fits a balanced workload. The default relational choice: JSONB, full-text search, strong integrity, huge ecosystem.
hosting	Google Cloud (Cloud Run + Cloud SQL)	First-class support for the chosen framework(s). Cloud Run scales containers to zero; Mumbai/Delhi regions; full compliance coverage.
supporting: search	PostgreSQL full-text search	Built-in tsvector search; plenty for moderate datasets, no extra service to run.
supporting: realtime	Socket.IO (self-hosted)	WebSocket layer with rooms and fallbacks; you scale it yourself (Redis adapter).
supporting: payments	Razorpay	UPI, cards and subscriptions for India; PCI scope stays with the PSP.
supporting: storage	Google Cloud Storage	Object storage with India regions.
supporting: auth	Firebase Authentication	Managed auth with phone OTP and social providers; generous free tier.
supporting: cache	Redis	In-memory cache for hot reads, sessions and rate limits.
supporting: queue	BullMQ (Redis-based)	Background jobs on top of Redis; no extra broker for Node teams.
supporting: other	React Native (Expo) mobile app	Web frontend does not cover the required mobile platforms; Expo ships Android/iOS from one JavaScript/TypeScript codebase, reusing the backend API.
supporting: cdn	Cloudflare	Free-tier CDN, caching, WAF and DDoS protection.
supporting: ci_cd	GitHub Actions	CI/CD next to the code; free tier covers small teams.

supporting: observability	Sentry (error tracking)	Error monitoring with a free tier; self-hostable if needed.

The drivers: web + Android platforms with SEO-critical public pages call for server-side rendering, so Next.js wins the frontend (React Native via Expo covers the provider Android app from the same TypeScript skill set). Realtime chat between customers and providers makes WebSocket support decisive in the backend score, so NestJS beats Django and FastAPI, and Socket.IO is the realtime layer. DPDP plus India data residency restrict hosting to providers with Indian regions; Cloud Run gives a three-person team without DevOps a managed path, and Razorpay covers UPI payments. PostgreSQL handles the relational core and its built-in full-text search covers provider search at 50 GB without operating Elasticsearch.

6.2 Walkthrough: P4 - QuickCredit (digital lending, India)

Status: **ok** - Modular monolith architecture: Angular frontend, Java (Spring Boot) backend, PostgreSQL database on AWS (EC2/ECS + RDS). Estimated cost: $287-$1434/month.

Layer	Choice	Why (from engine output)
frontend	Angular	Matches team language(s): typescript; well suited to fintech. Full framework with strong conventions and DI; favoured in enterprise teams.
backend	Java (Spring Boot)	Matches team language(s): java; well suited to fintech. Enterprise standard with deep ecosystem; heavier to run and slower to start with.
database	PostgreSQL	Well suited to fintech; fits a balanced workload. The default relational choice: JSONB, full-text search, strong integrity, huge ecosystem.
hosting	AWS (EC2/ECS + RDS)	First-class support for the chosen framework(s). The most complete cloud: Mumbai/Hyderabad regions, every compliance programme; needs real ops skill.
supporting: payments	Razorpay	UPI, cards and subscriptions for India; PCI scope stays with the PSP.
supporting: storage	AWS S3	Object storage with India regions; the default for uploads at any scale.
supporting: auth	Rolled JWT/OAuth in the backend	Framework-native auth (Django auth, Devise, Passport, etc.). No third-party dependency.
supporting: ml	Self-hosted open models (Ollama/vLLM on GPU VM)	Keeps data in your own infrastructure; needs a GPU and real ops.
supporting: cache	Redis	In-memory cache for hot reads, sessions and rate limits.
supporting: queue	AWS SQS + SNS	Managed queues, pay per request; zero broker operations.
supporting: other	React Native (Expo) mobile app	Web frontend does not cover the required mobile platforms; Expo ships Android/iOS from one JavaScript/TypeScript codebase, reusing the backend API.
supporting: cdn	AWS CloudFront	AWS-native CDN with India edge locations.
supporting: ci_cd	GitHub Actions	CI/CD next to the code; free tier covers small teams.
supporting: observability	Prometheus + Grafana	Self-hosted metrics and dashboards; full control, some ops.

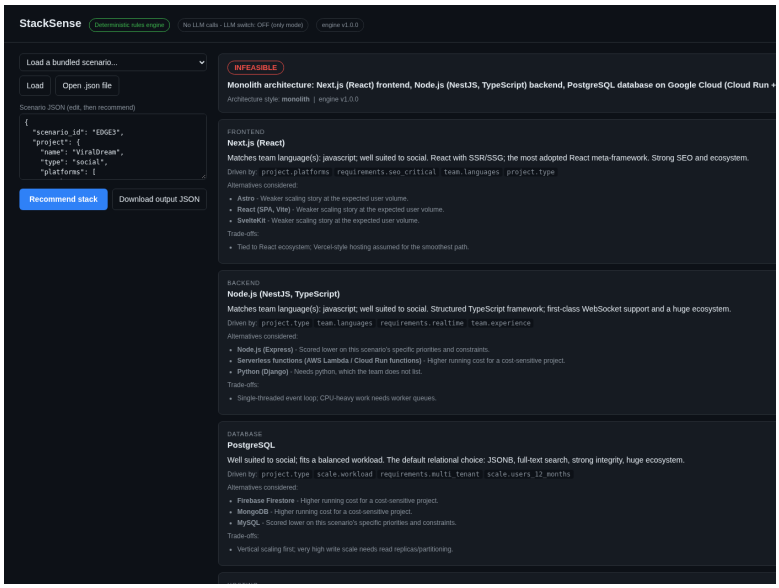
The drivers: RBI localisation plus PCI DSS, DPDP and ISO 27001 make residency and compliance the binding constraints - only AWS, GCP and Azure clear all of them, and AWS (Mumbai) wins on the senior team's priorities. A double-entry ledger demands strong transactional integrity, so PostgreSQL is unambiguous. Java/Spring Boot matches the team's languages and the domain's audit expectations. KYC documents stay in the project's own India-region infrastructure: authentication is rolled in the backend rather than a foreign SaaS, object storage is S3 (Mumbai), and because borrower data must not leave compliant infrastructure, ML inference is self-hosted open models on a GPU VM instead of a US-hosted

LLM API. Angular covers the web console while React Native ships the customer Android/iOS app.

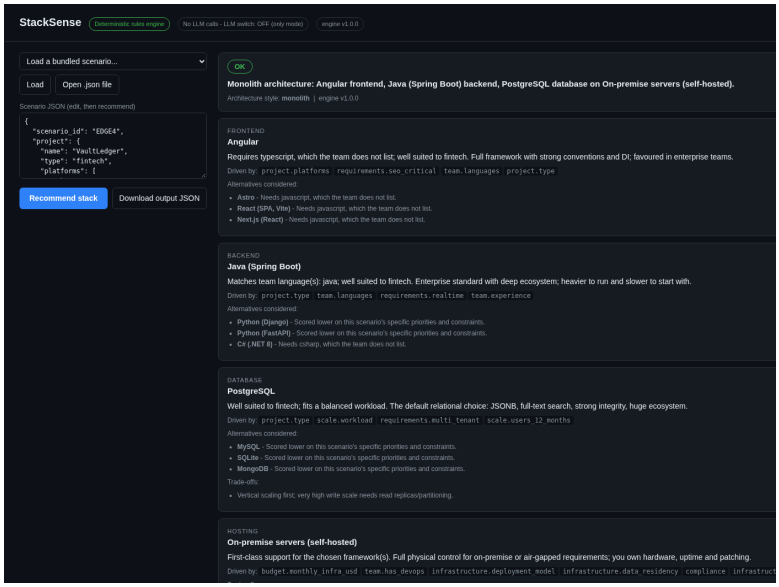
7. Edge-case handling

Each category from the brief, how the engine detects it, what it does, and the test scenario (all in source/tests/edge_scenarios/; run tests/run_edge_cases.py to reproduce):

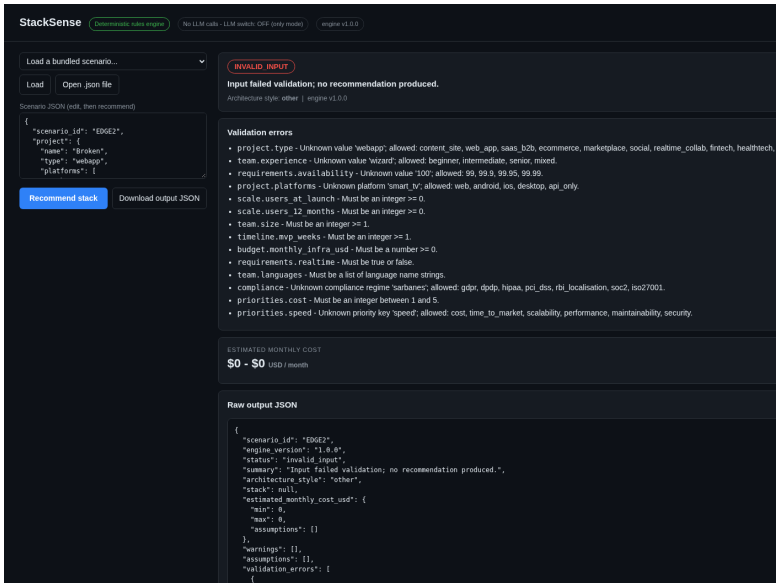
Category	Detection	Engine behaviour	Test
Missing fields	Required-field check plus documented defaults	invalid_input listing every missing required field; optional gaps filled with defaults recorded in assumptions	EDGE1
Invalid values	Type, range and enum validation per field	14 field-level errors at once for one broken file; never crashes	EDGE2
Contradictions	Cross-field rules (scale vs budget, availability vs team, timeline vs platforms)	Critical warnings; infeasible status with the relaxed constraint named	EDGE3
Over-engineering traps	Scale-appropriateness penalties; tiny-tool rule	A 120-user internal tool gets Django + SQLite + free-tier hosting, no microservices, no Kubernetes	EDGE5
Regulatory constraints	Hard filters on residency and compliance	RBI localisation forces India regions; auth, storage and ML stay in compliant infrastructure	P4, EDGE4
Existing systems	Deprecated-technology registry lookup	AngularJS, MySQL 5.7 and jQuery in existing_systems each raise a migration warning	EDGE6
Restricted deployment	Self-hostable-only filtering	Air-gapped bank: on-premise hosting, Keycloak SSO, MinIO storage, GitLab CI, Prometheus - zero SaaS	EDGE4
Skill mismatch	Language-match bonus/penalty in scoring	Non-listed languages lose points and the trade-off is named in the justification	P4 backend
Deprecated technology	Never in the candidate pool; registry for existing systems	Cannot be recommended; flagged when present in existing systems	EDGE6



EDGE3 - 2M users on $0/month: status infeasible, critical SCALE_BUDGET_CONFLICT and AVAILABILITY_TEAM_MISMATCH warnings, relaxed constraint stated in assumptions.



EDGE4 - air-gapped bank: every component self-hostable; on-premise hosting, Keycloak, MinIO, GitLab CI, Prometheus + Grafana.



EDGE2 - invalid input: 14 field-level validation errors returned together, status invalid_input, no stack produced.

8. Validation

Recommendations were sanity-checked three ways. (1) Against real-world reference architectures: the P2 output (Next.js + NestJS + PostgreSQL + managed cloud + Razorpay) matches the published stacks of comparable Indian marketplaces; the P4 output mirrors the AWS-Mumbai + PostgreSQL + PSP pattern Indian lending startups document in engineering blogs. (2) Against the judges' stated criteria: hard constraints were verified scenario by scenario - no recommended hosting lacks a required region or compliance programme - and every output includes driven_by fields, alternatives and trade-offs. (3) Mechanically: each public scenario was run twice with byte-identical results (determinism), the six edge scenarios above were run and inspected, and the engine was fed deliberately broken input to confirm it reports errors instead of crashing.

9. Limitations and future work

The knowledge base is curated by hand, so it ages: pricing and compliance programmes change, and new frameworks appear. Attribute scores encode my judgement and would benefit from calibration against a larger set of judged scenarios. The cost model is deliberately coarse (bands, not quotes) and does not model egress-heavy workloads well. The engine does not read the free-text description field beyond

validation; keyword-based parsing into suggested structured fields is the natural next step and can be added without touching the selection logic. Multi-region active-active designs and team growth over time are not modelled. Finally, 'one codebase' mobile companions are recommended as supporting components rather than being scored as full frontend alternatives.

10. References and third-party tools

The engine itself uses no third-party libraries, datasets or APIs (Python standard library only). Knowledge-base attributes were curated from these public sources:

• Stack Overflow Developer Survey 2025 - technology adoption. https://survey.stackoverflow.co/2025/

• DB-Engines Ranking - database models and popularity. https://db-engines.com/en/ranking

• AWS Well-Architected Framework. https://aws.amazon.com/architecture/well-architected/

• AWS pricing and compliance programmes. https://aws.amazon.com/pricing/ - https://aws.amazon.com/compliance/programs/

• Google Cloud pricing and compliance. https://cloud.google.com/pricing - https://cloud.google.com/security/compliance/offerings

• Microsoft Azure pricing and compliance. https://azure.microsoft.com/pricing/ - https://learn.microsoft.com/azure/compliance/

• Vercel, Netlify, Render, Railway, Fly.io, Firebase, Hetzner pricing pages (vercel.com, netlify.com, render.com, railway.com, fly.io, firebase.google.com, hetzner.com)

• Framework documentation: react.dev, nextjs.org, nuxt.com, svelte.dev, angular.dev, astro.build, docs.expo.dev, flutter.dev, nodejs.org, nestjs.com, fastapi.tiangolo.com, djangoproject.com, go.dev, spring.io, dotnet.microsoft.com, laravel.com, rubyonrails.org

• Database documentation: postgresql.org, mysql.com, mongodb.com, sqlite.org, redis.io, aws.amazon.com/dynamodb, firebase.google.com/docs/firestore

• Supporting services: meilisearch.com, elastic.co, algolia.com, rabbitmq.com, kafka.apache.org, socket.io, ably.com, pusher.com, supabase.com, stripe.com, razorpay.com, cashfree.com, auth0.com, keycloak.org, min.io, cloudflare.com, sentry.io, prometheus.io, grafana.com

• RBI - Storage of Payment System Data directive (6 April 2018). https://www.rbi.org.in/Scripts/NotificationUser.aspx?Id=11244

• Digital Personal Data Protection Act, 2023 (India). https://www.meity.gov.in/content/digital-personal-data-protection-act-2023

AI tools used while building are declared in Section 11.

11. AI assistance declaration

I used Instinct (an AI assistant agent) as a coding assistant while building this project: it helped draft engine code, the knowledge-base attribute structure, the web UI markup and this report's layout, and it ran the test scenarios. Every design decision - the input parameters and their defaults, the hard/soft constraint split, the scoring weights and bonuses, the knowledge-base contents and attribute scores, the cost model, and the final stack logic - was reviewed, tested and accepted by me, and I take responsibility for the reasoning in this report. As declared in Section 5, the engine itself makes no LLM calls: stack selection is entirely rule-based and deterministic, and the mandated LLM-off test passes in every configuration.